Project Design Phase
Problem – Solution Fit Template

Date	30.03.2026
Team ID	69bf8031c1eab70902258949
Project Name	Enforcing Mandatory Fields Using UI Policies and Migrating Changes with Update Sets
Maximum Marks	

- # Problem Statement

Issue: Users are submitting records (e.g., Incident, Change, or Custom Table) without providing critical data, leading to incomplete reporting and broken downstream automation.

Requirement: Specific fields must become **Mandatory** only under certain conditions (e.g., when a state changes) without writing complex Client Scripts.

- # Solution Overview: UI Policies

UI Policies provide a "no-code" way to manage field attributes. Unlike Data Policies, which enforce rules at the server level, UI Policies offer immediate feedback on the browser.

1. Configuration Steps

- **Table:** Select the target table (e.g., `incident`).
- **Conditions:** Define when the policy should run (e.g., `State | is | Resolved`).
- **Global:** Check this if it should apply to all views.
- **On Load:** Check this to ensure the policy runs when the form opens, not just when values change.

2. UI Policy Actions

Once the condition is met, create **UI Policy Actions** for the specific fields:

- **Mandatory:** Set to `True`.
- **Visible/Read Only:** Adjust as needed to guide the user.

- # Migration Strategy: Update Sets

To move these changes from **Development** to **Production**, you must use an Update Set to capture the configuration.

Phase 1: Capturing the Change

- **Navigate** to `Local Update Sets`.
- **Create New:** Name it clearly (e.g., `INC_MandatoryFields_v1`).
- **Submit and Make Current:** Ensure the "State" is `In Progress`.

- **Perform the Work:** Create your UI Policy and Actions. ServiceNow automatically tracks these "Customer Updates."

Phase 2: Closing and Exporting

- **Review:** Open your Update Set and look at the **Customer Updates** related list to ensure your UI Policy and Actions are listed.
- **Complete:** Change the State to `Complete`.
- **Export:** Click the **Export to XML** related link (if moving manually) or prepare for a "Pull" from the target instance.

• Best Practices & Common Pitfalls

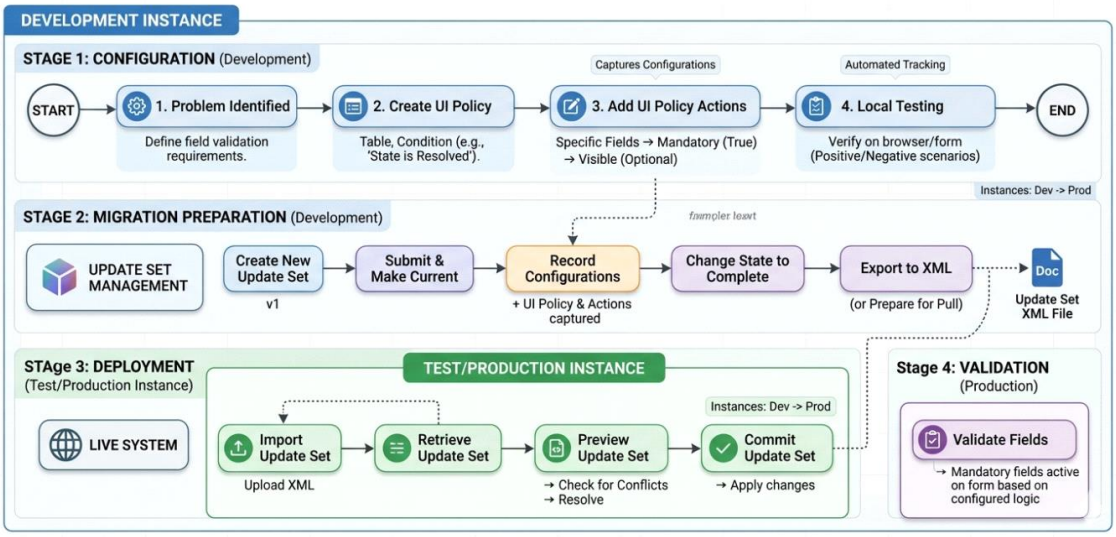
Feature	Best Practice
Naming Conventions	Be descriptive. Use: `Mandate [Field] when [Condition].`
Reverse if False	Keep this checked! It automatically un-mandates the field if conditions are no longer met.
Conflict Check	Ensure no existing Client Scripts or other UI Policies are fighting over the same field.
Update Set Scope	**Never** change your Update Set to `Complete` until you are 100% sure the logic works in Dev.

• Testing Protocol

Before marking the Update Set as complete, verify the following:

- **Positive Test:** Trigger the condition. Does the field turn mandatory (red asterisk)?
- **Negative Test:** Change the condition back. Does the mandatory requirement disappear?
- **Submission Test:** Try to save the form without the field filled. The system should prevent the save and highlight the field.

Note: UI Policies only enforce UI-level requirements. If you need to enforce mandatory fields for API integrations or imports, you should use a **Data Policy** instead.

ENFORCING MANDATORY FIELDS & MIGRATING CHANGES WORKFLOW

servicenow